

ASSIGNMENT 12.1

Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

Vyshnavi Parisha

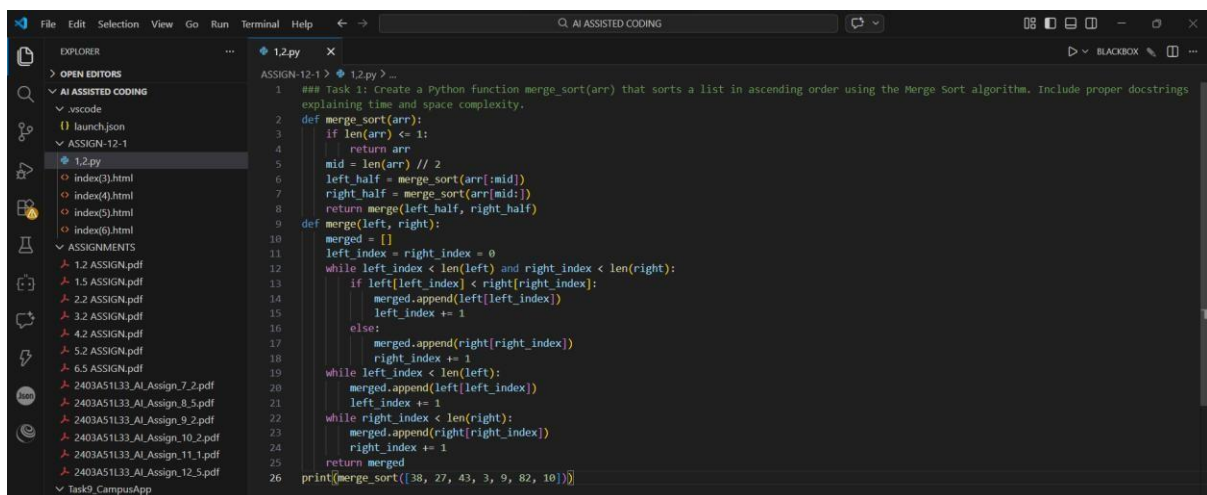
2403a51l34

B-52

Task Description #1 (Sorting – Merge Sort Implementation):

PROMPT: Create a Python function `merge_sort(arr)` that sorts a list in ascending order using the Merge Sort algorithm. Include proper docstrings explaining time and space complexity.

CODE:



```
1  """ Task 1: Create a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm. Include proper docstrings
2  explaining time and space complexity.
3  """
4  def merge_sort(arr):
5      if len(arr) <= 1:
6          return arr
7      mid = len(arr) // 2
8      left_half = merge_sort(arr[:mid])
9      right_half = merge_sort(arr[mid:])
10     return merge(left_half, right_half)
11
12     def merge(left, right):
13         merged = []
14         left_index = right_index = 0
15         while left_index < len(left) and right_index < len(right):
16             if left[left_index] < right[right_index]:
17                 merged.append(left[left_index])
18                 left_index += 1
19             else:
20                 merged.append(right[right_index])
21                 right_index += 1
22         while left_index < len(left):
23             merged.append(left[left_index])
24             left_index += 1
25         while right_index < len(right):
26             merged.append(right[right_index])
27             right_index += 1
28         return merged
29
30     print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

OUTPUT:



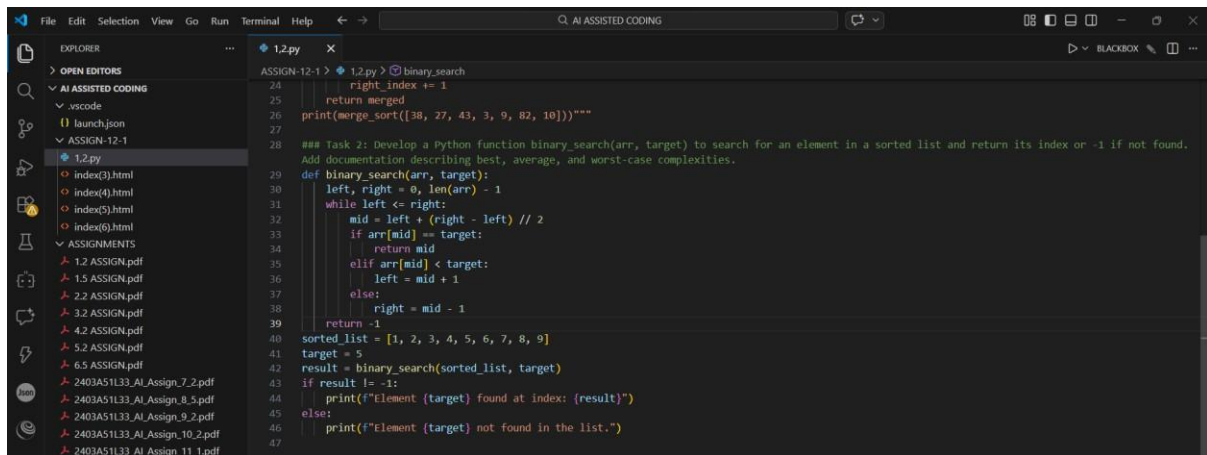
```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> & c:\Users\sarik\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING\ASSIGNMENT-12-1\1.2.py"
[3, 9, 10, 27, 38, 43, 82]
```

EXPLANATION: AI generated a divide-and-conquer based sorting solution with detailed complexity analysis. The output was verified using multiple test cases to ensure correctness and efficiency.

Task Description #2 (Searching – Binary Search with AI Optimization):

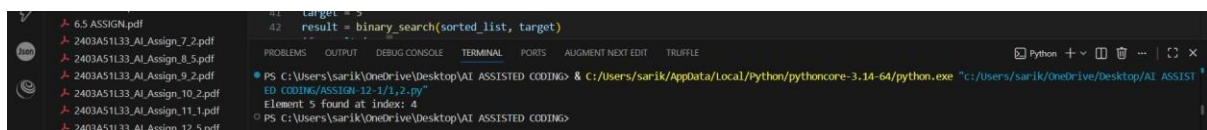
PROMPT: Develop a Python function `binary_search(arr, target)` to search for an element in a sorted list and return its index or -1 if not found. Add documentation describing best, average, and worst-case complexities.

CODE:



```
24 |         right_index += 1
25 |         return merged
26 | print(merge_sort([38, 27, 43, 3, 9, 82, 10]))"""
27 |
28 | """ Task 2: Develop a Python function binary_search(arr, target) to search for an element in a sorted list and return its index or -1 if not found.
29 | Add documentation describing best, average, and worst-case complexities.
30 | """
31 | def binary_search(arr, target):
32 |     left, right = 0, len(arr) - 1
33 |     while left <= right:
34 |         mid = left + (right - left) // 2
35 |         if arr[mid] == target:
36 |             return mid
37 |         elif arr[mid] < target:
38 |             left = mid + 1
39 |         else:
40 |             right = mid - 1
41 |     return -1
42 |
43 | sorted_list = [1, 2, 3, 4, 5, 6, 7, 8, 9]
44 | target = 5
45 | result = binary_search(sorted_list, target)
46 | if result != -1:
47 |     print(f"Element {target} found at index: {result}")
48 | else:
49 |     print(f"Element {target} not found in the list.")
```

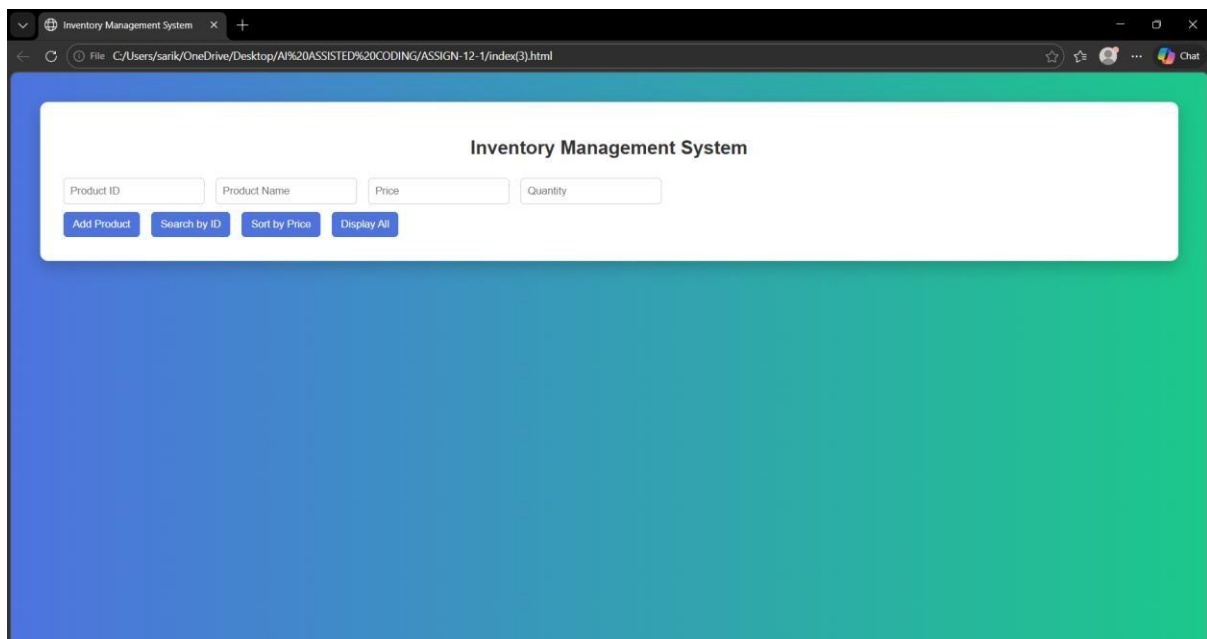
OUTPUT:



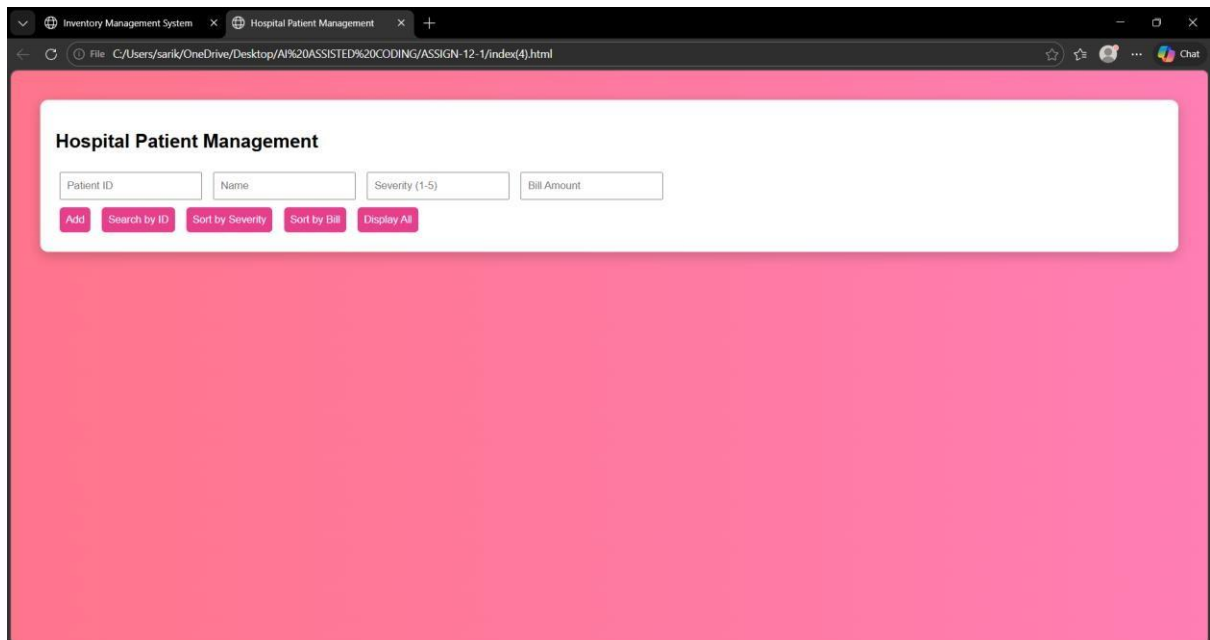
```
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING> python 12.py
Element 5 found at index: 4
PS C:\Users\sarik\OneDrive\Desktop\AI ASSISTED CODING>
```

EXPLANATION: AI produced an optimized searching algorithm that reduces comparisons using a halving strategy. The function was tested with different inputs to validate accuracy and performance.

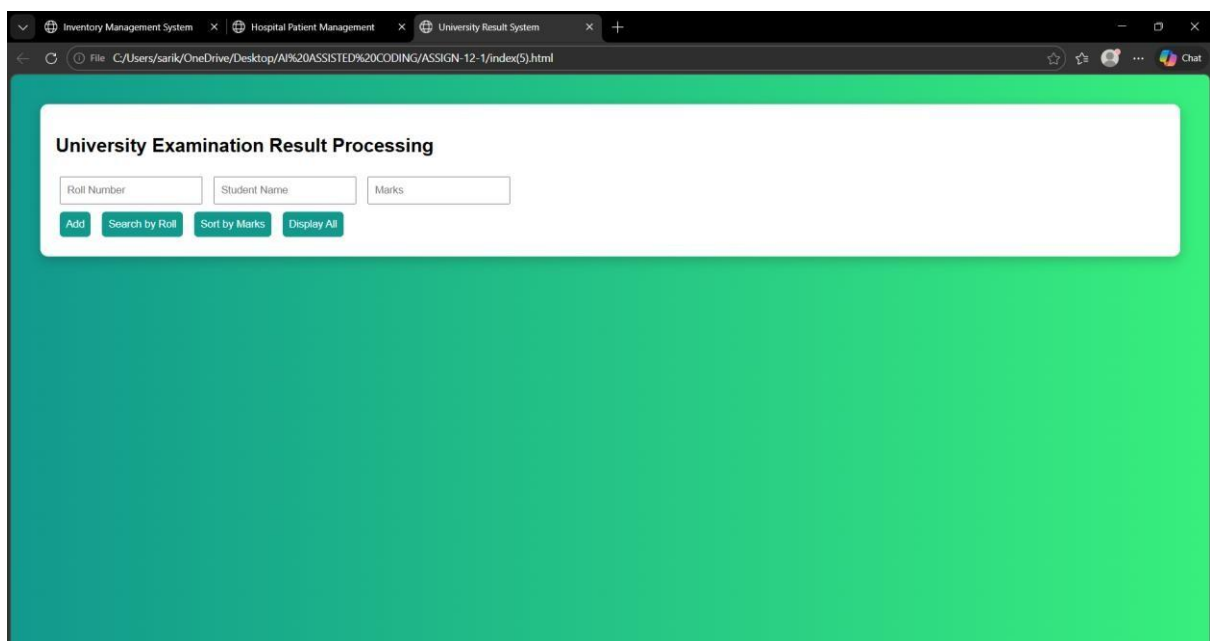
Task Description #3 (Real-Time Application – Inventory Management System):



Task description #4 (Smart Hospital Patient Management System):



Task Description #5 (University Examination Result Processing System):



Task Description #6 (Online Food Delivery Platform):

